QuantumPay

Quantum-Safe Payment Integrity via Quantum Walk Hash Functions

March 25, 2026

The Problem

India's Payment Infrastructure at Risk

- 17+ billion UPI transactions per year
- All protected by **SHA-256** — invented 2001
- NIST confirmed in 2024: quantum computers running **Grover's Algorithm** halve SHA-256's effective security
- 256-bit security $\rightarrow$ 128-bit effective

Consequence

An attacker with a quantum computer can find two different payment amounts that produce the **same hash**. Payment fraud becomes undetectable.

Our Solution

QuantumPay

Replace SHA-256 with a **Quantum Hash Function (QHF)** based on Controlled Alternate Quantum Walks (CAQW) — running on normal hardware today.

Property	SHA-256	**QHF (CAQW)**
Quantum resistant	No	Yes
Avalanche effect	Yes	Yes
Runs on normal CPU	Yes	Yes
Based on	Prime math	Quantum walks
Hash length	256 bits	512 bits

The Math — Coin Operators

Three coin operators steer the quantum walker:

$$\hat{C}_i = \begin{pmatrix} \cos\theta_i & \sin\theta_i \\ \sin\theta_i & -\cos\theta_i \end{pmatrix}$$

- Message bit $= 0 \rightarrow$ apply $\hat{C}_0$
- Message bit $= 1 \rightarrow$ apply $\hat{C}_1$
- Steps exceed message $\rightarrow$ apply $\hat{C}_2$

Example: $\hat{C}_1$ with $\theta_1 = \pi/6$

$$\hat{C}_1 = \begin{pmatrix} 0.866 & 0.5 \\ 0.5 & -0.866 \end{pmatrix} \quad \Rightarrow \quad 0.866^2 + 0.5^2 = 1.0 \checkmark$$

The Math — Shift and Evolution

Shift operators move the walker on the grid:

$$\hat{S}_x : \ \text{coin} = 0 \Rightarrow x + 1 \bmod N, \quad \text{coin} = 1 \Rightarrow x - 1 \bmod N$$

One full evolution step (right-to-left order):

$$|\psi\rangle \xrightarrow{\hat{C}_i} \xrightarrow{\hat{S}_x} \xrightarrow{\hat{C}_i} \xrightarrow{\hat{S}_y} |\psi'\rangle$$

Probability at each grid position after r steps:

$$P(x, y) = \sum_{c \in \{0,1\}} \left| \langle x, y, c \,|\, \psi \rangle_r \right|^2$$

The Math — Hash Generation

Convert the $N \times N$ probability matrix to hash bits:

$$h_{x,y} = \left\lfloor P(x,y) \times 10^8 \right\rfloor \bmod 256$$

$$\text{hash} = h_{0,0} \,\|\, h_{0,1} \,\|\, \cdots \,\|\, h_{N-1,N-1}$$

Total hash length $= N^2 \times 8$ bits. For $N = 8$: **512 bits**.

Avalanche Property (Section 6.1.2, paper)
Flipping 1 bit in input changes $\approx 50\%$ of hash bits:

$$\delta = \frac{1}{512} \sum_{k=1}^{512} \text{hash}(M)_k \oplus \text{hash}(M')_k \approx 0.45$$

Demonstrated live: 232/512 bits changed (45.31%)

Blockchain Payment Integrity

Each block is linked using QHF:

$$\text{block\_hash}_i = \text{QHF}(\text{index} \parallel \text{payment\_id} \parallel \text{amount} \parallel \text{prev\_hash}_{i-1})$$

Verification condition:

$$\text{QHF(received payment data)} \stackrel{?}{=} \text{stored payment\_id}$$

Tamper cascade — attacker modifies Block *i*:

$$\text{QHF(tampered data)} \neq \text{stored hash} \quad \Rightarrow \quad \text{REJECTED}$$

Even if attacker recomputes Block i's hash:

$$\text{new hash}_i \neq \text{prev\_hash}_{i+1} \quad \Rightarrow \quad \text{chain broken from Block } i \text{ onwards}$$

Live Demo Results

Test	Result
QHF hash of “hello”	512-bit hash, 4096 steps
Avalanche (1-bit flip)	232/512 bits changed (45.31%)
SHA-256 avalanche	128/256 bits changed (50%)
Chain verification	All 5 blocks valid
Attack detection	Block #2 mismatch caught instantly
Tamper cascade	Chain broken from Block #2 onwards

Research Foundation

Abd El-Latif et al. (2021). *Quantum-Inspired Blockchain-Based Cybersecurity.* Information Processing and Management, 58(3), 102549.

Architecture

Frontend
Send Payment | Payment Ledger
QHF Explorer | How It Works

↓ HTTP / fetch API ↑

quantum_hash.py	payment_chain.py
CAQW simulation	In-memory blockchain
QHF, Avalanche	verify_chain()
SHA-256 compare	tamper_payment()

main.py | FastAPI
/payments/new | /payments/verify
/qhf/hash | /qhf/avalanche | /qhf/compare

Impact and Next Steps

What We Built
A working quantum-safe payment integrity system using real quantum walk mathematics — running on normal hardware, implementable today, referenced to published research.

Next Steps

- Integrate with real UPI sandbox API
- Increase N from 8 to 15 for full paper specification (1800-bit hash)
- Add CAQW encryption layer (Algorithm 1 from the paper) for full confidentiality
- Benchmark on cloud infrastructure

Source code: https://github.com/tveshas/Quantum-Blockchain-lol